

RELATÓRIO DE LABORATÓRIO

Gerência do Processador no SOsim

Escalonamento circular, prioridades estáticas e prioridade dinâmica

Aluno: Marcelo Flávio de Carvalho Frota Porto
Data: 15 de setembro de 2026

Fortaleza — CE

1. Introdução

Este relatório analisa quatro experimentos de escalonamento de processos propostos no documento “SOsim – Laboratório de Gerência do Processador”. O SOsim é uma ferramenta educacional que permite visualizar processos, mudanças de estado, escalonamento e memória virtual. O foco desta atividade é comparar o comportamento de processos CPU-bound, que fazem uso intenso do processador, e I/O-bound, que alternam pequenos surtos de CPU com períodos de espera por entrada e saída.

O escalonamento circular (Round Robin) distribui a CPU em fatias de tempo. Ao final do quantum, o processo em execução pode ser preemptado e colocado novamente na fila de prontos. Em políticas com prioridade, um processo pronto de maior prioridade pode assumir o processador antes de outro de menor prioridade.

Observação metodológica: como o arquivo fornecido contém o roteiro, mas não contém o arquivo de execução, capturas de tela ou medições produzidas no SOsim, os tempos numéricos apresentados abaixo são valores ilustrativos e coerentes com o comportamento esperado. Eles não devem ser apresentados como leitura instrumental real. As conclusões qualitativas respondem integralmente às questões do roteiro.

2. Objetivos

- Comparar o escalonamento circular sem prioridade e com prioridades.
- Examinar o efeito da variação do quantum.
- Avaliar o impacto das prioridades sobre processos CPU-bound e I/O-bound.
- Entender a vantagem do mecanismo adaptativo de prioridade dinâmica.

3. Conceitos utilizados

Conceito	Descrição
CPU-bound	Permanece por longos períodos usando a CPU e tende a consumir todo o quantum disponível.
I/O-bound	Usa a CPU em surtos curtos e frequentemente passa de execução para espera de E/S; ao concluir a E/S, retorna ao estado pronto.
Quantum	Tempo máximo contínuo de CPU concedido a um processo no escalonamento circular.
Preempção	Retirada do processador de um processo antes de sua conclusão, por fim do quantum ou chegada de processo mais prioritário.
Prioridade dinâmica	Prioridade ajustada durante a execução conforme o comportamento do processo, reduzindo dependência de uma classificação fixa.

4. Exercício 1 — Escalonamento circular sem prioridade

Configuração: escalonamento circular; dois processos com a mesma prioridade, sendo P1 CPU-bound e P2 I/O-bound; observação proposta de três minutos.

Com a mesma prioridade, os processos são atendidos pela ordem da fila de prontos. P1 normalmente utiliza todo o quantum e retorna ao fim da fila. P2 executa por pouco tempo, solicita E/S e fica bloqueado; quando a E/S termina, retorna à fila de prontos. Por isso, igualdade de prioridade não implica igualdade de tempo acumulado de CPU: o I/O-bound não está sempre pronto para executar.

Cenário ilustrativo	P1 CPU-bound	P2 I/O-bound	Interpretação
Quantum médio	≈ 118 s de CPU	≈ 24 s de CPU	P2 passa boa parte do período esperando E/S; P1 aproveita as lacunas.
Quantum maior	≈ 126 s de CPU	≈ 20 s de CPU	Menos trocas, porém maior espera até P2 recuperar a CPU.
Quantum menor	≈ 110 s de CPU	≈ 25 s de CPU	Resposta mais rápida, com maior frequência de trocas de contexto.

Pergunta: o que acontece se o quantum aumentar ou diminuir?

Resposta: quando o quantum aumenta, cada processo pode permanecer mais tempo contínuo na CPU. Isso reduz o número de trocas de contexto e o overhead, mas piora o tempo de resposta dos processos que aguardam na fila; no limite, o comportamento se aproxima do FIFO. Quando o quantum diminui, há maior alternância e melhor responsividade, porém cresce o custo das trocas de contexto. Com processos sempre prontos e de mesma prioridade, o balanceamento continua aproximadamente justo ao longo do tempo, mas a eficiência cai se o quantum for pequeno demais.

Mudanças de estado observáveis: P1 alterna principalmente entre Pronto e Execução; P2 alterna com maior frequência entre Pronto, Execução e Espera/Bloqueado.

5. Exercício 2 — Prioridades estáticas: I/O-bound mais prioritário

Configuração: P1 CPU-bound com prioridade 3 e P2 I/O-bound com prioridade 4, considerando que o maior número representa maior prioridade, conforme sugerido pela sequência do roteiro.

Sempre que P2 termina uma operação de E/S e volta ao estado Pronto, sua prioridade superior permite preemptar P1. O processo I/O-bound recebe atendimento rápido, inicia logo o próximo ciclo de E/S e melhora o aproveitamento concorrente de CPU e dispositivos. P1 continua usando a maior parcela do tempo total de CPU, pois P2 permanece bloqueado durante suas operações de E/S.

Processo	Prioridade	Tempo ilustrativo de CPU	Comportamento
P1 CPU-bound	3	≈ 112 s	Executa nos intervalos em que P2 aguarda E/S; pode ser preemptado quando P2 fica pronto.

Processo	Prioridade	Tempo ilustrativo de CPU	Comportamento
P2 I/O-bound	4	≈ 29 s	Recebe a CPU rapidamente ao retornar da E/S e apresenta menor latência.

Comparação com o Exercício 1

No exercício sem prioridade, P2 precisa aguardar sua posição na fila. Com prioridade superior, P2 volta da E/S e é atendido mais rapidamente. Portanto, a principal diferença não é necessariamente uma grande inversão no tempo total acumulado de CPU, mas a redução do tempo de resposta do I/O-bound e o aumento da quantidade de ciclos CPU-E/S completados.

Pergunta: o que acontece se o tempo de espera do processo I/O-bound aumentar ou diminuir?

Resposta: se o tempo de espera por E/S aumentar, P2 ficará bloqueado por períodos maiores, acumulará menos tempo de CPU e provocará menos preempções; P1 aproveitará mais o processador. Se a espera diminuir, P2 retornará mais vezes à fila de prontos e, por ser mais prioritário, preemptará P1 com maior frequência, elevando seu número de execuções e reduzindo a continuidade do CPU-bound.

6. Exercício 3 — Prioridades estáticas: CPU-bound mais prioritário

Configuração: P1 CPU-bound com prioridade 4 e P2 I/O-bound com prioridade 3.

Nessa configuração, o CPU-bound permanece pronto quase todo o tempo e possui prioridade superior. Assim, ele tende a ocupar sucessivamente o processador. Mesmo quando P2 conclui a E/S e volta ao estado Pronto, poderá aguardar até P1 bloquear, terminar ou perder a CPU segundo as regras específicas do simulador. O resultado é maior latência para P2 e risco de inanição se houver fluxo contínuo de processos mais prioritários.

Pergunta: quais devem ser os critérios para determinar as prioridades?

Resposta: as prioridades devem refletir os objetivos do sistema: criticidade e prazos; necessidade de resposta interativa; frequência e duração dos surtos de CPU; comportamento de E/S; importância do serviço; tempo já aguardado; e prevenção de inanição. Em geral, processos interativos e I/O-bound recebem tratamento mais rápido porque usam a CPU por pouco tempo e liberam o processador logo em seguida. Processos CPU-bound podem receber prioridade menor sem serem impedidos de progredir. Também convém aplicar envelhecimento (aging), elevando gradualmente a prioridade de quem espera por muito tempo.

Conclusão do exercício: atribuir prioridade alta apenas a um CPU-bound pode prejudicar a responsividade global. A prioridade não deve ser escolhida somente pela importância aparente do processo, mas pelo equilíbrio entre resposta, produtividade, justiça e utilização dos recursos.

7. Exercício 4 — Prioridade dinâmica e mecanismo adaptativo

Configuração: prioridade dinâmica; P1 CPU-bound e P2 I/O-bound iniciando com a mesma prioridade.

O mecanismo adaptativo identifica indiretamente o perfil pelo uso da CPU. P1 tende a consumir o quantum completo e pode ter sua prioridade reduzida ou deixar de receber benefícios. P2 bloqueia antes de esgotar o quantum e, ao retornar da E/S, tende a receber atendimento rápido. Dessa forma, o comportamento adequado emerge durante a execução, sem depender exclusivamente de prioridades fixas atribuídas no momento da criação.

Comparação com o Exercício 2

No Exercício 2, o benefício do I/O-bound depende de uma classificação estática correta: prioridade 4 para P2 e 3 para P1. No mecanismo dinâmico, ambos podem começar iguais e o sistema ajustar o tratamento conforme o padrão efetivamente observado. Isso torna o escalonador mais flexível diante de processos que mudam de fase ou apresentam intensidades distintas de E/S.

Pergunta: qual é a vantagem para processos I/O-bound de perfis diferentes?

Resposta: a prioridade dinâmica adapta o atendimento à frequência e à duração dos surtos de cada processo. Um I/O-bound que retorna com frequência pode ganhar resposta rápida, enquanto outro com longas esperas não mantém artificialmente uma prioridade elevada quando está bloqueado. O mecanismo atende melhor diferenças reais de comportamento, reduz a latência, favorece a sobreposição entre CPU e E/S e evita que uma prioridade estática inadequada prejudique o sistema.

8. Síntese dos resultados

Política	Principal efeito	Risco ou custo
Circular sem prioridade	Distribuição em rodízio entre processos prontos.	Quantum pequeno aumenta overhead; quantum grande piora resposta.
Prioridade estática — I/O alto	Boa resposta para E/S e melhor sobreposição de recursos.	Depende de classificação correta.
Prioridade estática — CPU alto	Favorece processamento intensivo.	Pode atrasar I/O-bound e causar inanição.
Prioridade dinâmica	Adapta-se ao comportamento efetivo.	Maior complexidade e necessidade de regras de ajuste equilibradas.

9. Conclusão

Os experimentos mostram que o escalonamento não busca apenas dividir a CPU igualmente, mas equilibrar eficiência, tempo de resposta e justiça. O quantum regula a relação entre responsividade e custo de troca de contexto. Prioridades estáticas podem melhorar o atendimento de processos I/O-bound, mas uma escolha inadequada pode favorecer processos CPU-bound e prejudicar a interatividade. A prioridade dinâmica é mais adaptável, pois ajusta o tratamento com base no comportamento durante a

execução. Assim, a melhor política é aquela que mantém o processador ocupado, atende rapidamente processos curtos e interativos e impede que processos de menor prioridade fiquem indefinidamente sem executar.

10. Referências

MAIA, Luiz Paulo. SOsim: Simulador para o Ensino de Sistemas Operacionais. Training. Disponível no site oficial do SOsim. Acesso em: 15 set. 2026.

UNIVERSIDADE FEDERAL DE MINAS GERAIS. Sistemas Operacionais — Escalonamento. Material didático do Departamento de Ciência da Computação. Acesso em: 15 set. 2026.

UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO. Processos (3) — Escalonamento. Material didático. Acesso em: 15 set. 2026.

SOsim — Laboratório de Gerência do Processador, setembro de 2003. Documento fornecido para a atividade.